
4.1 Introduction

Of the data structures looked at so far, the list is the least structured. Unlike stacks where items can only be pushed or popped from the top and queues which can only have items enqueued at the rear or dequeued from the front, a *list* can have items inserted or deleted from the front, rear, or anywhere in between. As such, it might sound as if the list might be easier to work with, but all the possibilities can make the list a little more complicated to implement.

There are two types of lists, unordered and ordered. Unordered lists can have the items stored randomly within them. Although this makes it easy to insert an item into a list, it can make it difficult to find an item. For example, imagine a desk in which all the papers on it are not organized in any particular fashion. If a particular paper needed to be found, then the search would have to start at one end of the papers on the desk and keep on searching until the paper being searched for was found. In the best case, it would be the first paper examined and in the worst case, it would be the last one examined. On average, if there were n papers on the desk then it would take $n/2$ times to find the paper being searched for, or more formally the search process would be of order n or $O(n)$.

Clearly it would be nice to shorten this search time, so it would be beneficial to have the papers in some sort of order to reduce the search time. For example, the papers on the desk could be placed in alphabetical order and then a binary search could be used which could cut the time needed significantly. As a reminder, an ordered stack of papers could be cut into two halves, then the paper to be found would be in one of the two halves and the other half would not need to be searched. This process could then be repeated on the half the paper should be in, where that half could be cut in half, and so on. The process of dividing the stack in half repetitively until the paper is found is logarithmic. If there were 64 papers, and each time the pile of papers was cut in half there would be 32, 16, 8, 4, and 2 papers until there was finally 1 paper which means that there would only be six comparisons. As a reminder, 2^6 is 64 and conversely the $\log_2 64$ is 6. So although with a sequential

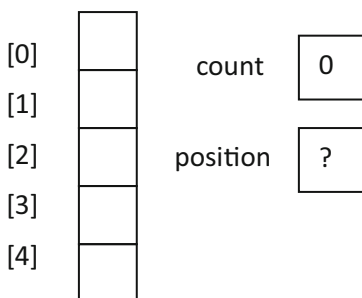
search the search time is of $O(n)$, with a binary search the time is of $O(\log n)$. Clearly it is beneficial to use an ordered list over an unordered list should there need to be a lot of searching.

One way of creating a list is to input all the items into the list in a random order and then organize them into the proper order using a sort algorithm. The bubble or selection sort that might have been learned about in a previous course or text could be used, or the insertion or quick sort as discussed in Chap. 9 of this text could be used. However, often all the items for a list are not immediately available and are entered sporadically. Although a new item could be inserted randomly and one of the above sorting algorithms could be used to put the new item in the correct location, the use of a sort algorithm to put one or a few items in the proper location could be considered excessive.

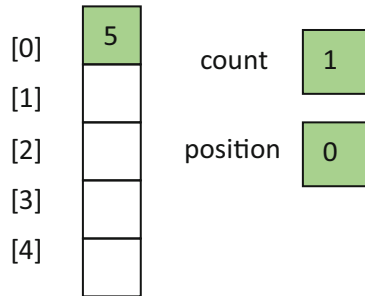
Instead, it is helpful to define the list to be able to put individual items into the proper location as they are entered and not rely on a sort when a simpler algorithm could be used in the first place. This method of inserting an item correctly initially is comparable to students turning in a homework assignment in a professor's office. Instead of students just laying the papers on the desk and the professor going back later and sorting the papers, the professor could insert the homework assignment in the correct position alphabetically in the pile of assignments as they are turned in. That way in case the pile of papers needs to be searched before all the assignments have been turned in, the pile is in the correct order. This method of inserting items in the correct location will be discussed below and it is a variation of the insertion sort which will be discussed in Chap. 9.

4.2 Analysis and Design

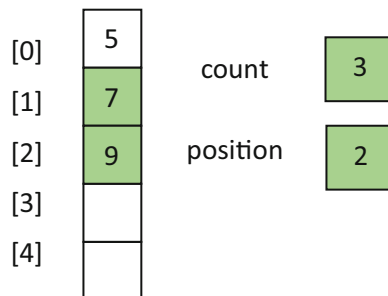
Again using a smaller array so that it is easier to see how a list works, yet large enough so that patterns can be seen, assume the existence of a five-element array, a position that indicates where an item should be inserted or removed, and a count of the number of items in the list. Initially the list would be empty as indicated by the count being 0.



When the first number, for example the integer 5, is inserted into the list, it is placed at position 0 since there are no numbers in the list and the count is incremented to 1 where the changes are shown in green below:

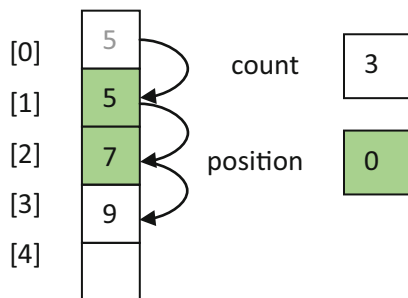


When the next number is inserted into the list, say 7, then the list needs to be searched for the correct position for the number to be inserted, the number is inserted into location 1, and the count is incremented. Likewise with the number 9, where the list is again searched, the number is inserted into the list at position 2, and count is incremented as reflected below:

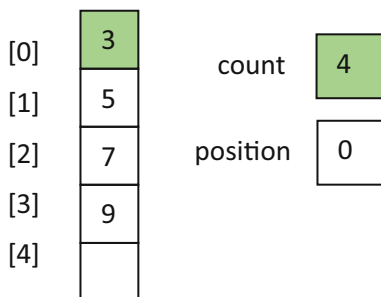


Note that the list can be searched using either a sequential search $O(n)$ or a binary search $O(\log n)$. When the list is small a binary search would not be very helpful, but if the list were very large, then the savings in time would be significant. Since the list here is small, it will be assumed that the sequential search will be used for the time being and a binary search is left as an exercise at the end of the chapter.

So far, the numbers that have been inserted were in order, but what would happen if a number such as 3 were to be inserted? As might be suspected, all the numbers in the list would need to be moved down in the list to make room for the number 3 in position 0.

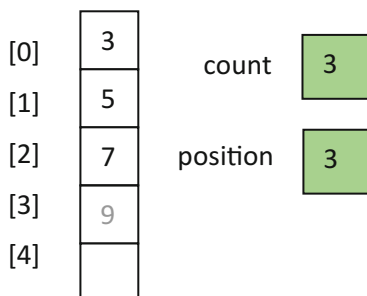


As before, note above that the number 5 is light gray to indicate that it was merely copied from position 0 to 1. When the number 3 is inserted, the number 5 is overlaid and count is incremented by 1 as shown below:



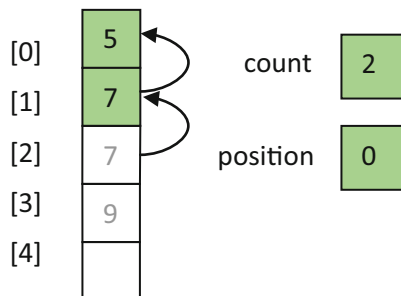
The result is that the moving of n items in a list would be $O(n)$ which is something that has been seen in the last two chapters on stacks and queues. Unfortunately, this is a disadvantage of inserting an item into an ordered list that has been implemented using a simple array, and will be discussed more in detail later.

So far numbers have only been inserted into a list. What if the number 9 were to be deleted from the list? Again, the list would need to be searched, the number removed, and the count decremented by one.



As with stacks and queues previously, note that the number 9 above is now gray indicating that it is no longer part of the list but still present in the array. Again, this illustrates that the list is distinct from the array in which it is implemented.

Removing a number from the end of the list does not seem to be a problem, but what if a number is to be removed from the middle or beginning of the list? For example, what if the number 3 were to be removed? Again, the problem of having to move all the numbers in the list occurs but in reverse from when the number was inserted and the count is decremented.



Again note above that the number 7 in grey is still in the array, since it was only copied from position 2 to 1. However, as before the count has been decremented to indicate that the 7 in position 2 is no longer part of the list. Of course, when another item is inserted, the number will be overwritten.

The problem of moving the numbers occurs when inserting or deleting items from the middle or beginning of the list and is difficult to avoid when implementing an ordered list in a simple array. One solution when deleting an item is to just put in a number that is not part of the potential data set, so that if the list is searched or output it is just ignored. However, this could cause problems in trying to find a unique number to use. An alternative is to include another dimension in the array or a field in an object in the array that indicates when an item has been deleted. However, this can waste space to include an extra field and might pose a problem when a large number of items have been deleted where the deleted items might actually take up more space in the array than there are actual items in the list.

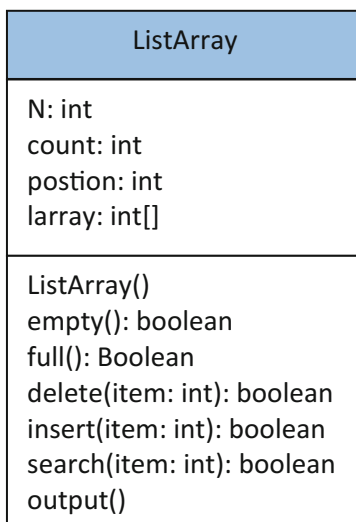
As will be seen in the next chapter, there is a solution to the problem of having to move items around in an array which will speed up the insertion and deletion of items. The solution is that of a linked list which can be implemented using references and objects. However, linked lists are not without their own problems concerning searching and memory utilization as will be seen later. For now, the inefficiency of inserting and deleting will just be an inconvenience, since there is much that can be learned from implementing a list using an array.

4.3 ListArray Class: Data Members and Methods

Similar to the data members in previous two chapters on stacks and queues, the data members needed for the `ListArray` class can be seen in the previous section. An array will be needed which can be called `larray` and it can be of size `N`. Another variable needed is `position` which indicates where an item will be inserted or deleted. The last variable needed is `count` which indicates how many items are in the list and as before can be helpful when determining whether the list is full or empty.

Of course there will need to be public methods for `insert` and `delete`, where `insert` will have a parameter for the number to be inserted called `item` and will return a `boolean` value to indicate whether the number in `item` was inserted. The `delete` method will also have a parameter called `item` containing the number to be deleted, and there will also be a `boolean` value returning method to indicate whether the number was deleted. As before, there will need to the `boolean` value returning `full` and `empty` methods which could be `public` or `private` as necessary. Alternatively, as with stacks and queues, a method can be included for increasing the size of the array should the list become full. Further, as mentioned in the previous section, there also needs to be a `search` method. It should have a parameter `item` containing the number to be searched for, would return a `boolean` to indicate whether the `item` was found, `update position` where the number was found, and it could be `public` or `private` as needed. Lastly, a method to output the contents of the list called `output` is also helpful.

The following UML diagram summarizes what is necessary:



4.3.1 The Insert Method

It is probably easiest to start with the `insert` method because it will be used to create a list. As indicated in the UML diagram, it accepts an `item` as a parameter to be inserted and returns a `boolean` value to indicate whether the `item` was inserted or not. In the basic skeleton for the method, the `boolean` variable `inserted` is declared locally and initialized for the default case of `false`.

```
public boolean insert(int item) {  
    boolean inserted=false;  
    //code for insert  
    return inserted;  
}
```

To fill in the skeleton, it can be assumed that if the list is full, then a number would not be able to be inserted and the default value of `false` is returned. Since it would seem that the list would usually not be full, the message for a full list is included in the `else` section of the `if` statement as shown below:

```
if(!full())  
    //code for inserting item in list  
else  
    System.out.println("List is Full");
```

As before, the `println` could be replaced with an exception, but since the code for the test program in the next section will need to continue processing when the list is full, the `println` statement is used. Further, since the `full` and `empty` methods from the `QueueArray` class used in Sect. 3.3 utilize `count`, they can be copied into the current `ListArray` class as will be seen later in Fig. 4.1.

Before writing the code to insert an item in the list, it should be determined whether duplicate values will be allowed. For example, in the case where there is more than one person with the same last name in a classroom, duplicates could be allowed. However, in the current case duplicate numbers will not be allowed and code for duplicates is left as an exercise at the end of the chapter.

The result is that the list should be searched to see if the item is already in the list before it is inserted. As shown previously in the UML diagram, the `search` method is sent an `item` and returns a `boolean` indicating whether the item has been found or not. If the number in `item` is not found, it should be inserted; otherwise it should not be inserted and again the default value is returned.

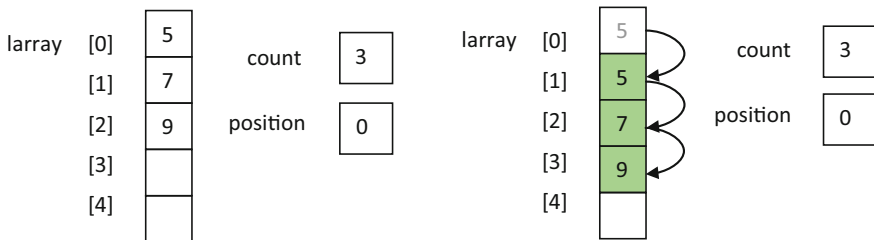
```

if(!full())
    if(!search(item)) {
        // code for insert
    }
else
    System.out.println("List is Empty");

```

If the number in `item` is not found and it is to be inserted in the list, the location or position in the list needs to be known. Since the `search` method can only return one piece of information, in this case whether the `item` was found or not, the location in the array where the `item` should be inserted can be stored in the data member `position` which will be accessible by both the `search` and `insert` methods. There are other ways to accomplish this, but for now this technique allows the code to be less complicated.

However, unless an item is being inserted into the last location in the list, it will be necessary to move all the items following the `position` further down in the list. For example, given the following scenario from the previous section shown on the left below, assume that the number 1 is to be inserted into `larray[0]`, then all the numbers would need to be copied down further in the array. The number in `larray[2]` needs to be copied to `larray[3]`, then the number in `larray[1]` can be copied to `larray[2]` and then the number in `larray[0]` can be copied to `larray[1]` as shown on the right below:



In trying to see the pattern it sometimes helps to list the indexes in a table as follows:

```

larray[2] → larray[3]
larray[1] → larray[2]
larray[0] → larray[1]

```

Notice that the locations of the data being copied are from `larray[2]` through `larray[0]` to `larray[3]` through `larray[1]`. Although a loop could be written with an index, say `j`, to go from 3 to 1, that would not be very useful under differing circumstances since there might not always be exactly three items to copy.

Instead, note that `count` also contains a 3 and that would be a good starting point. Although in this instance the value decreases until it hits 1, this constant would not be very useful depending on where in the list a number were to be inserted. Instead note that the `position` to be inserted is at 0, so the value of `j` could be decremented as long as it is greater than the `position` to be inserted. Lastly, for each iteration of the loop the numbers in the list need to be moved from one less than the `j`th position into the `j`th position. In other words, from `larray[j-1]` to `larray[j]` as shown in the following segment:

```
for(int j = count; j > position; j--)
    larray[j] = larray[j-1];
```

In the case of inserting an item at the beginning or end of the list, and after the location in `larray` is available, then `item` should be copied into that `position`, the variable `count` incremented by 1, and the `inserted` flag should be set to `true` as follows:

```
larray[position] = item;
count++;
inserted = true;
```

What a number was to be inserted at the end of the list? In this case, none of the numbers would need to be moved. Does this mean that a separate `if` statement is needed prior to the `for` loop to accommodate this special case? For example, should an `if` statement such as the following be inserted prior to the `for` statement?

```
if(position == count)
```

Although it might seem that this is necessary, recall from a previous class or text, what type of iteration structure is the `for` loop? Yes, it is a fixed iteration loop, but it is also a pretest loop which means its test is at the beginning of the loop and the body of the loop is executed zero to many times. If the `for` loop is written correctly, the body of the loop should never execute in this case. For example in the previous drawing, if the number `item` 11 is to be inserted, then `search` should return the value 3 for `position` where the number 11 should be inserted. The initial value of `j` in the `for` loop would then take on the value 3 from `count` and when `j` is compared to `position`, the value 3 in `j` will not be greater than the value 3 in `position`, so the loop will not iterate, and thus the `if` statement is not needed. Then as before, the value in `item` will be put into `larray[3]` indicated by `position`, the variable `count` will be incremented by 1, and the `inserted` flag would be set to `true`.

Putting all of the above code together results in the following method:

```
public boolean insert(int item) {
    boolean inserted=false;
    if(!full())
        if(!search(item)) {
            for(int j=count; j>position; j--)
                larray[j] = larray[j-1];
            larray[position] = item;
            count++;
            inserted = true;
        }
    else
        System.out.println("List is Full");
    return inserted;
}
```

As an aside, notice in the above code segment that there is a set of braces in the then section of the outer `if` statement. The question is, should the braces be included? At first glance, one might answer no, because there is only one statement, the nested `if` statement, and the outer braces would not appear to be necessary. If the outer `if` was only an if-then structure and not an if-then-else structure, this would be correct. However, this is an example of a nested if-then-if structure. Thinking back to one's first Java course or text, what is one of the problems with a nested if-then-if structure? Recall that they are susceptible to the "dangling else" problem, where an `else` always matches up with the closest `if` and without the enclosing braces, the `else` above would match up with the inner `if` instead of the outer `if` which is not what is intended and results in a logic error. The alternative is to rewrite the if-then-if as an if-then-else-if structure, but as mentioned previously the list will probably tend not to be full, so the error message is placed further down in the method in the else section instead.

4.3.2 The Delete Method

Having created the `insert` method, it is convenient to examine the `delete` method before designing the `search` method, since the `search` method will be used by both methods. As with `insert`, examining the UML diagram provides the skeleton for the `delete` method.

```
public boolean delete(int item) {  
    boolean deleted = false;  
    //code for delete  
    return deleted;  
}
```

Again, the number to be deleted is sent in the parameter `item` and the method returns a boolean value indicating whether the number was deleted or not, where the boolean variable `deleted` is set to `false` as the default.

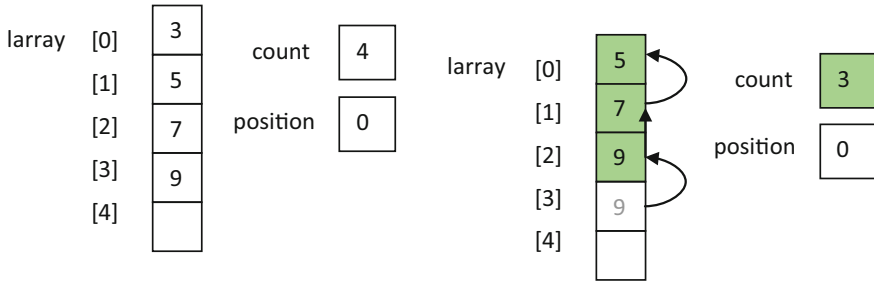
Opposite to `insert`, which is concerned with whether the list is full, the `delete` method checks to ensure that the list is not empty before attempting to delete a number. If the list is empty, then a message is output (or alternatively an exception can be thrown) and the default value of `false` will be returned.

```
if(!empty())  
    // code for delete  
else  
    System.out.println("List is Empty");
```

If the list is not empty, then the list will need to be searched for the number to be deleted and if the number is found then it can be deleted.

```
if(!empty())  
    if(search(item)) {  
        // code for delete  
    }  
else  
    System.out.println("List is Empty");
```

The act of deleting is merely moving the other numbers in the list to take up the spot once occupied by the number in `item` as shown in these figures modified from Sect. 3.2.



Again, following the type of analysis used in Sect. 4.3.1, note that if the number 3 were to be deleted, then `search` would return the `position` as 0. Then all of the other numbers in the list would need to be copied down in the list to take up the space vacated by the number 3. The number in `larray[1]` will be copied to `larray[0]`, the number in `larray[2]` will be copied to `larray[1]`, and the number in `larray[3]` will move to `larray[2]` as shown below:

```
larray[1]→larray[0]
larray[2]→larray[1]
larray[3]→larray[2]
```

A variable, again say `j`, could start at `larray[0]`, but what if the number to be deleted was at a location other than 0? As before, note that the `search` method would return the location of the number in the data member `position`. In this case it would be 0, and is a good place to start the loop control variable `j`. Then the number at location `j + 1` would need to be moved into location `j` for each iteration of the loop. Since `count` is originally 4 as shown above, the loop would iterate while `j` is less than or equal to `count-2`. Alternatively, the `for` could be rewritten as `j<count-1` as follows:

```
for(int j=position; j<count-1; j++)
    larray[j] = larray[j+1];
```

Once all the numbers have been copied, then `count` would need to be decremented by 1 and the `deleted` flag would be set to `true`:

```
count--;
deleted = true;
```

Putting all the above pieces together for the complete `delete` method, the result would be:

```
public boolean delete(int item) {
    boolean deleted=false;
    if(!empty())
        if(search(item)) {
            for(int j=position; j<count-1; j++)
                larray[j] = larray[j+1];
            count--;
            deleted = true;
        }
    else
        System.out.println("List is Empty");
    return deleted;
}
```

4.3.3 The Search Method

Having created both the `insert` and `delete` methods, it is now time to create the `search` method. Starting again with the UML diagram, the skeleton of the method can be written

```
private boolean search(int item) {
    boolean found = false;
    // code for search
    return found;
}
```

As with `insert` and `delete`, the `search` method is sent a number in `item` to be searched for, and will return a `boolean` value whether it was found or not with the initial value as `false`. As mentioned earlier, to keep it simple, a sequential search will be used. The `position` in which the number might be found can be initialized to 0 and while `position` is less than the `count` of the number of items in the list, the loop will continue to iterate by incrementing the `position` counter. Further, the `item` being searched for can be compared to each location in the list. If the number in `item` is found, the `found` flag can be set to `true` as shown below:

```
position = 0;
while(position < count) {
    if(larray[position] == item)
        found = true;
    position++;
}
```

The above code would work for either an unordered or an ordered list. However, once a number is found, is there a reason to continue looking at the other numbers in the list? No, there isn't, and although stopping the loop in the average case will not make it an order of magnitude faster since $n/2$ is still $O(n)$, it would help speed the algorithm up some and it is worth adding the code to stop the loop once the item is found as shown below:

```
position = 0;
while(position < count && !found) {
    if(larray[position] == item)
        found = true;
    position++;
}
```

So in order to delete a found item the above code would work. However, what if the item was not in the list? In an unordered list the loop would need to continue and check all of the items in the list just as with an unordered pile of papers on a desk. In an ordered list once the loop had gone past the spot where the item should have been, there is no reason to continue searching for the item. Again, if the pile of papers on the desk are in alphabetical order, and the name being searched for began with the letter R, then once one reached a last name beginning with the letter S, there would be no reason to continue looking for the paper.

Unfortunately, the above code only stops when the number in `item` is found and not when one has gone past where the number should be located in an ordered list. So instead of stopping the loop only when the number is found, it should also stop when the number has been found or when it has gone past the location where the number should have been. Since the `found` flag is used to indicate only when a number is found, it should not be used to control the `while` loop. Instead another `boolean` variable could be used to indicate the above two conditions when the loop should stop. As a result, the `boolean` variable `stop` is added to the local variables in the method and is also initialized to the default value of `false` as well. The resulting code is as follows:

```
position = 0;
while(position < count && !stop) {
    if(larray[position] >= item)
        stop = true;
    position++;
}
```

Note the use of the variable `stop` instead of the variable `found` and that the `if` statement has changed from `==` to `>=`. This will now stop the loop when either the location of the number is found or when it goes past where the number should have been found. Although this helps for the case when it is not found, the code segment needs to be further modified to distinguish between these two possibilities. This can be accomplished by adding a second `if` statement inside the existing `if` statement. Once it is known that either the number has been found or has gone past where it should have been found, the second `if` statement checks if it is equal and sets the `found` flag to `true`.

```
position = 0;
while(position < count && !stop)
    if(larray[position] >= item) {
        stop = true;
        if(larray[position] == item)
            found = true;
        position++;
    }
```

In other words if the number is not in the list, `stop` will be set because the number is greater than or equal to one in the list. Further, since it is not equal, the `found` flag will not be set. On the other hand if it is equal, `stop` will still be set to terminate the loop and `found` will also be set to indicate it was in the list. However, notice that `position` is incremented in both cases. In the case where it is found, `position` will be pointing to the element after where it was found and if it is not in the list, `position` will be one beyond where it should be. How can this be solved?

Unfortunately, `position` is being incremented in both cases when it shouldn't be incremented. It should not be incremented when the loop has ceased iterating when `stop` has been set. The solution is that an `else` statement can be paired with the outer `if` statement so that it only increments when the loop continues to search for the correct location as shown below:

```
position = 0;
while(position < count && !stop)
    if(larray[position] >= item) {
        stop = true;
        if(larray[position] == item)
```

```
        found = true;
    }
    else
        position++;
```

As in previous methods in this section, note the careful use of the braces to ensure that the `else` is paired with the outer `if` and not accidentally creating a dangling `else` problem by being paired with the inner `if` statement. Putting all the code together and including the declaration and default initialization of the boolean variable `stop`, the complete search method is as follows:

```
private boolean search(int item) {
    boolean found=false, stop=false;

    position = 0;
    while(position < count && !stop)
        if(lArray[position] >= item) {
            stop = true;
            if(lArray[position] == item)
                found = true;
        }
        else
            position++;
    return found;
}
```

4.3.4 Complete ListArray Class

In addition to the constructor, and the `empty` and `full` methods, the only remaining method is `output` which outputs the list and is fairly simple to implement. All four of these are listed without explanation in the complete `ListArray` class in Fig.4.1.

4.4 Simple Test Program

As in the previous chapter a simple test program should be created. Similar to the queue program in the previous chapter, this program will continue prompting from a sentinel controlled loop until a command of S for Stop is entered by the user. If the user enters an I for Insert, the user is prompted for a number to be inserted, and upon return from the `insert` method a message is output indicating whether or not the number was inserted. Similarly, should the user enter a D for Delete, the

```
public class ListArray {

    static private final int N=5;
    private int position, count, larray[];

    public ListArray() {
        count=0;
        larray = new int[N];
    }

    private boolean empty() {
        return count<=0;
    }

    private boolean full() {
        return count>=N;
    }

    public boolean delete(int item) {
        boolean deleted=false;
        if(!empty())
            if(search(item)) {
                for(int j=position; j<count-1; j++)
                    larray[j] = larray[j+1];
                count--;
                deleted = true;
            }

        else
            System.out.println("List is Empty");
        return deleted;
    }

    public boolean insert(int item) {
        boolean inserted=false;
        if(!full())
            if(!search(item)) {
                for(int j=count; j>position; j--)
                    larray[j] = larray[j-1];
                larray[position] = item;
                count++;
                inserted = true;
            }

        else
            System.out.println("List is Full");
        return inserted;
    }

    private boolean search(int item) {
        boolean found=false, stop=false;
        position = 0;
        while(position < count && !stop)
            if(larray[position] >= item) {
                stop = true;
                if(larray[position] == item)
                    found = true;
            }
    }
}
```

Fig. 4.1 ListArray Class

```

        }
    else
        position++;
    return found;
}

public void output() {
    System.out.print("List: ");
    int j = 0;
    while(j != count) {
        System.out.print(larray[j]+" ");
        j++;
    }
    System.out.println();
}
}

```

Fig. 4.1 (continued)

user is then prompted for the number to be deleted, and upon return from the `delete` method a message is output indicating whether the number was deleted or not. A command of O for Output invokes the `output` method, where the contents of the list are output to the screen. Lastly, should the user enter some other letter, a message is output to indicate to try again. As before, both upper and lower case letters are allowed for the commands.

```

import java.util.*;
class Ch3Test {
    public static void main(String[] args) {

        Scanner scanner;
        scanner = new Scanner(System.in);

        char response;
        int number;

        ListArray list;
        list=new ListArray();

        System.out.println();
        System.out.print("Do you wish to I = Insert, D = Delete, " +
            "O = Output, or S = Stop: ");
        response = scanner.next().charAt(0);
        while(response!='S' && response!='s') {
            if (response=='I' || response=='i') {
                System.out.print("Enter a number to insert into " +
                    "the list: ");
                number = scanner.nextInt();
                if(list.insert(number))

```

```

        System.out.println("The number was inserted");
    else
        System.out.println("The number was not inserted");
    }
    else
        if(response=='D' || response=='d') {
            System.out.print("Enter a number to delete from " +
                "the list: ");
            number = scanner.nextInt();
            if(list.delete(number))
                System.out.println("The number was deleted");
            else
                System.out.println("The number was not deleted");
        }
        else
            if(response=='O' || response=='o')
                list.output();
            else
                System.out.println("The command entered is not "
                    + "I, D, O, or S, please try again.");
        System.out.println();
        System.out.print("Do you wish to I = Insert, D = Delete, "
            + "O = Output, or S = Stop: ");
        response = scanner.next().charAt(0);
    }
    System.out.println();
    System.out.println("End of Program");
    System.out.println();
}
}

```

Sample output is shown below indicating all the prompts and many of the messages generated by the main program as well as some of the messages indicating whether the list is full or empty from the `ListArray` class:

```

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: I
Enter a number to insert into the list: 1
The number was inserted

```

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List: 1

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 3
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 5
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 7
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 9
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List: 1 3 5 7 9

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 11
List is Full
The number was not inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 2
The number was not deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 9
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List: 1 3 5 7

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 1
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 3
The number was deleted

```
Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 5
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 7
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List:

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: i
Enter a number to insert into the list: 11
The number was inserted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: o
List: 11

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 11
The number was deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: d
Enter a number to delete from the list: 11
List is Empty
The number was not deleted

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: q
The command entered is not I, D, O, or S, please try again.

Do you wish to I = Insert, D = Delete, O = Output, or S = Stop: s

End of Program
```

4.5 Complete Program: Checking Opcodes in Assembly Language

In this section a program that simulates an assembler is developed to determine whether all the opcodes in an assembly language program are valid. Assembly language is a low-level language that consists of mnemonics instead of ones and zeros used in machine language programs. In order for a program to be executed on a machine, programs written in a high-level language can be converted to an

Fig. 4.2 Assembly code

Symbol	Opcode	Operand	

SAMPLE	START	FIRST	Line 1
THOMAS	WORD	3	Line 2
SYLVESTER	WORD	7	Line 3
CATS	WORD	?	Line 4
FIRST	LOAD	THOMAS	Line 5
	ADD	SYLVESTER	Line 6
	STORE	CATS	Line 7
	STOP		Line 8
	END		Line 9

assembly language program by a compiler and then converted to a machine language program by an assembler. A program written in an assembly language that implements the following Java code might look like the one in Fig. 4.2.

```
int thomas = 3, sylvester = 7, cats;
cats = thomas + sylvester;
```

Essentially, the program adds two numbers 3 and 7 that are in the variables THOMAS and SYLVESTER respectively, and stores the result 10 in the variable CATS. The column labels, the line below the column labels, and line numbers are not part of the program but included to show how the words line up. In order from left to right, the three fields of an instruction are the symbol, the operation code (opcode), and the operand. The first field is typically reserved for the names of variables or labels for instructions. The second field is typically used for opcodes that represent executable instructions and also assembler directives which tell the assembler what to do. The third field is used for operands.

The first line indicates the start of the program. The symbol SAMPLE is the name of the program, the opcode START tells the computer when the program starts, and operand FIRST indicates the location of the first instruction. The second through fourth lines are directives that tell the assembler to reserve the memory locations. Line 2 reserves a space for the symbol THOMAS which is initialized to 3. Similar to the second line, the third line initializes the variable SYLVESTER to 7. The fourth line reserves an uninitialized memory location called CATS. Line 5 fetches the contents from the memory location THOMAS and stores it into a register. A register is a memory location in the CPU where among other things arithmetic can be performed. Then the ADD operation in Line 6 gets the value from the variable SYLVESTER and adds it to the contents of the register. The next line stores the value in the register into the memory location CATS. The STOP instruction halts the execution of the program. In the last line, the END directive indicates to the assembler the end of the program.

When the assembly code is translated into machine code, an assembler first checks if the program consists of valid opcodes. If there is a syntax error, the assembler will output an error message. This is the part of the assembler that will be developed. First, the list of valid opcodes called OpTable is created. A list of the

valid opcodes shown below are stored in a text file, `OpCode.txt`. They are listed in unordered manner in the file and will be inserted in alphabetical order in the list.

```
STORE
ADD
LOAD
WORD
DIV
STOP
SUB
END
START
MUL
```

With this application the items stored in the list are `String` objects. Therefore, the `ListArray` class shown in Fig. 4.1 would need to be modified for the `String` class. As discussed in Chap. 3, since rewriting the `ListArray` class for the specific type of object every time a new application is developed is not very efficient, instead the `ListArrayGeneric` class that can be used for any objects will be defined by replacing the `int` in `ListArray` by the type parameter `T`. Therefore the array, `larray` is of type `T` and item that will be inserted to the list is also of type `T`. Unlike the `StackArrayGeneric` and `QueueArrayGeneric` classes, `ListArrayGeneric` needs to compare items to determine where the new item should be inserted in the array. Unfortunately, comparing two items using the following statement does not work in the generic class.

```
larray[position] >= item
```

Since `larray[position]` and `item` are both of type `T` which are references to an object, instead of comparing the contents of the objects, the above statement compares the addresses of two objects. What is the solution to this problem? The solution is the `compareTo` method that is included in the `Comparable` interface could be used to compare two objects. The `Comparable` interface is generic and is used to order objects. It is found in the `Java.lang` package and contains only one method named `compareTo`. The method is used to compare contents of the object that calls the method and the contents of the specified object in the parameter. The `compareTo` method takes an object of any type and returns an integer. Specifically, it returns a positive integer, zero, or a negative integer if the object that is calls the method is greater than, equal to, or less than the specified object respectively. Now the above statement can be rewritten using the `compareTo` method as shown below, which indicates whether contents of the object that `larray[position]` references is greater than or equal to contents of the object that `item` references.

```
larray[position].compareTo(item) >= 0
```

First, the type parameter in the `ListArrayGeneric` class must support comparison with other instances of its own type. To accomplish this the class declaration will be:

```
public class ListArrayGeneric<T extends Comparable<T>>
```

indicating the type `T` inherits a `compareTo` method from the `Comparable` interface and the `compareTo` method will be implemented in the `ListArrayGeneric` class.

Instead of an array of the `Object` typecast into the generic type in the `StackArrayGeneric<T>` class as in Fig. 2.2 and the `QueueArrayGeneric<T>` class as in Fig. 3.2, an array of the `Comparable` type is created and then cast into the array of the generic type in the constructor of the `ListArrayGeneric` class as shown below. This is done because every `T` is a `Comparable` type as in the class declaration of the `ListArrayGeneric` class.

```
larray = (T[]) new Comparable[N];
```

The implementation of the `compareTo` method, which is the only method the `Comparable` interface contains, is added at the end of the class. This method will be used in the search method.

```
public int compareTo(T item) {
    int result;
    if (reference.compareTo(item) > 0)
        result = 1;
    else
        if (reference.compareTo(item) < 0)
            result = -1;
        else
            result = 0;
    return result;
}
```

Figure 4.3 shows the complete `ListArrayGeneric` class definition.

In the main program, two methods, `createOpTable` and `checkOpCodes` are called. The `createOpTable` method creates a list of opcodes. An unordered list of opcodes is read from a file and placed in alphabetical order in the list. The `checkOpCodes` method simulates an assembler. The main program without detailed implementation of the two methods is shown below and the two methods are defined shortly.

```

public class ListArrayGeneric<T extends Comparable<T>> {

    private final int N=20;
    private int position, count;
    private T[] lArray;
    T reference;

    public ListArrayGeneric() {
        count=0;
        larray = (T[]) new Comparable[N];
    }

    private boolean empty() {
        return count <= 0;
    }

    private boolean full() {
        return count >= N;
    }

    public boolean insert(T item) {
        boolean inserted=false;

        if(!full())
            if (!search(item)){
                for(int j=count; j>position; j--)
                    larray[j] = larray[j-1];
                larray[position] = item;
                count++;
                inserted = true;
            }

        else
            System.out.println("List is Full");
        return inserted;
    }

    public boolean delete(T item) {
        boolean deleted=false;
        if(!empty())
            if(search(item)){
                for(int j=position; j<count-1; j++)
                    larray[j] = larray[j+1];
                count--;
                deleted = true;
            }

        else
            System.out.println("List is Empty");
        return deleted;
    }

    public boolean search(T item) {
        boolean found, stop;
        found = false;
        stop = false;
        position = 0;
    }
}

```

Fig. 4.3 ListArrayGeneric<T> class

```

        while(position != count && !stop)
            if(larray[position].compareTo(item) >= 0){
                stop = true;
                if(larray[position].compareTo(item) == 0)
                    found = true;
            }
            else
                position++;
        return found;
    }

    public void output() {
        System.out.print("List: ");
        int j = 0;
        while(j != count) {
            System.out.print(larray[j]+" ");
            j++;
        }
        System.out.println();
    }

    public int compareTo(T item) {
        int result;
        if(reference.compareTo(item) > 0)
            result = 1;
        else
            if(reference.compareTo(item) < 0)
                result = -1;
            else
                result = 0;
        return result;
    }
}

```

Fig. 4.3 (continued)

```

import java.util.*;
import java.io.*;

class OpCodes {
    private static ListArrayGeneric<String> opTable
        = new ListArrayGeneric<String>();

    public static void main(String[] args) throws IOException {

        // create op table
        createOpTable();
        // check op codes in assembly code
        checkOpCodes();
    }
}

```

```
// create op table
public static void createOpTable() throws IOException {
    // defined shortly
}

// check op codes in assembly code
public static void checkOpCodes() throws IOException {
    // defined shortly
}
}
```

In the `createOpTable` methods, first the file `opCodes.txt` is opened for reading. It reads the contents of the file line by line until it reaches the end of the file. For each line it reads, it inserts the opcode into the list called `opTable` in alphabetical order. Notice that because it deals with a file which may not exist in the system, `throws IOException` is added for the system to handle such a situation. The complete definition of `createOpTable` method is given below:

```
public static void createOpTable() throws IOException {
    String inStr;
    Scanner inFile = new Scanner(new File("opCodes.txt"));
    while(inFile.hasNextLine()) {
        inStr = inFile.nextLine();
        opTable.insert(inStr);
    }
    inFile.close();
}
```

In the second method, `checkOpCodes`, after the program prints the column labels for the output, it opens the file `assemblyCode.txt` for reading. As it reads a line of assembly code inside the `while` loop, it outputs what it read and checks if the opcode of the instruction is valid. Although the opcode lies between column 11 and 20 of the line, the line without the operand ends before column 20. Therefore, two cases are examined; a line with an operand and a line without an operand. For the line with an operand, a substring between columns 11 and 20 is extracted from the original line. Because the actual opcode may not occupy the entire 10 character long space, the substring most likely contains trailing whitespace. That has to be removed before it is compared to the opcodes in the list, `opTable`. How can trailing whitespace in the string be removed? Among many useful methods already defined in the `String` class, the `trim` method is the one that can be used here. It returns a copy of the string with the whitespace from the beginning and end of the string removed. After checking the opcode, if the line does

not contain a valid opcode an error message will be output. Note that depending on whether or not there is an operand, the error message will need to be tabbed differently.

```
// check op codes in assembly code
public static void checkOpCodes() throws IOException {
    String inStr;
    int len;

    System.out.println("Symbol   Opcode   Operand   Error");
    System.out.println("-----");
    + ("-----");

    Scanner inFile = new Scanner(new File("assemblyCode.txt"));

    while(inFile.hasNextLine()) {
        inStr = inFile.nextLine();
        System.out.print(inStr);
        len = inStr.length();

        // line with operand
        if(len >= 20) {
            inStr = inStr.substring(10, 20);
            if(!opTable.search(inStr.trim()))
                System.out.print("\t\tInvalid Opcode "
                    + inStr.trim());
        }
        // line without operand
        else {
            inStr = inStr.substring(10, len);
            if(!opTable.search(inStr.trim()))
                System.out.print("\t\t\tInvalid Opcode "
                    + inStr.trim());
        }
        System.out.println();
    }
    inFile.close();
}
```

Using the assembly code shown in Fig. 4.2, which does not have any syntax errors, a slightly altered assembly language program is created as shown in Fig. 4.4. Note that WORD in the third line and END in the last line are changed to WARD and EMD respectively. It is saved in `assemblyCode.txt` in simple text format.

When the main program is compiled and executed using the new sample assembly code in Fig. 4.4, the output of the program looks as given below:

```
SAMPLE    START    FIRST
THOMAS    WORD     3
SYLVESTER WARD     7
CATS      WORD     ?
FIRST     LOAD     THOMAS
          ADD      SYLVESTER
          STORE    CATS
          STOP
          EMD
```

Fig. 4.4 Assembly code containing syntax errors, a variation of the code in Fig. 3.2

Symbol	Opcode	Operand	Error

SAMPLE	START	FIRST	
THOMAS	WORD	3	
SYLVESTER	WARD	7	Invalid Opcode: WARD
CATS	WORD	?	
FIRST	LOAD	THOMAS	
	ADD	SYLVESTER	
	STORE	CATS	
	STOP		
	EMD		Invalid Opcode: EMD

The output is a listing of the assembly language program and syntax error messages. Using the code in `assemblyCode.txt` shown in Fig. 4.4 there are two opcodes that are misspelled, WARD and EMD. Therefore, output indicates two syntax errors.

4.6 Summary

- Lists can be ordered or unordered.
- Although lists are easier to use because there are few restrictions as to where an item can be inserted or deleted, they can be complicated to implement.
- A sequential search can be used on both ordered and unordered list, whereas a binary search can be used only on an ordered list.
- Using a sequential search on an ordered list, once the search has gone past the location where an item should have been located, the search can stop. Although not an order of magnitude faster, it is faster than letting the search continue through to the end of the list.
- A count can be used to indicate whether a list is full or empty as was done with queues.
- Generic types make lists reusable with different types of objects.

4.7 Exercises (Items Marked with an * Have Solutions in Appendix B)

- *1. Assuming the list `intList` of type `ListArray` in Fig. 4.1 is initially empty, draw a list after walking through the following code segment:

```
intList.insert(41);
intList.insert(6);
intList.delete(5);
intList.delete(6);
intList.insert(78);
intList.insert(5);
intList.insert(79);
intList.delete(41);
intList.insert(5);
```

2. Assuming the list `characterList` of type `ListArray Generic<Character>` which is an object of the generic class `ListArrayGeneric<T>` in Fig. 4.3 is initially empty, draw a list after walking through the following code segment:

```
characterList.insert('U');
characterList.insert('V');
characterList.delete('U');
characterList.insert('W');
characterList.delete('A');
characterList.insert('P');
characterList.insert('Q');
characterList.delete('P');
characterList.insert('C');
```

- *3. Write a method `deleteAt` that can be added to the `ListArray` class in Fig. 4.1 which removes the item at the specific position from the list. The method accepts a position as a parameter.
4. Write a method `positionOf` that can be added to the `ListArray` class in Fig. 4.1 which returns the position of the specific item in the list. The method accepts an item as a parameter.
5. Write a method `search` that performs a binary search and that can replace the existing search method in Fig. 4.1 `ListArray` class.
6. Implement a `ListArray` class which allows duplicates in an ordered list. The method `search` should indicate the number of the occurrences of the item

being searched for, and all the duplicates should be deleted in the `delete` method.

7. Write a program to merge two lists of strings using `ListArrayGeneric<T>` in Fig. 4.3.

The resulting list should not include duplicates.

8. Write a program to find an intersection of two given sets of integers using `ListArrayGeneric<T>` in Fig. 4.3. For example, if two integer sets are $P = \{4, 7, 1, 10\}$ and $Q = \{4, 2, 5, 1, 3\}$, the intersection of these two sets is $P \cap Q = \{1, 4\}$.